



x32dbg 调试脚本编写手册

x64dbg-automate 客户端 API 与 MCP 调试工具
中文参考 高频命令完整代码示例 避坑速查

适用环境：x64dbg-automate 一键调试套装 Python 客户端 / WorkBuddy 连接器 双通道
2026 年 9 月



目录

1. 总览：两种驱动方式与环境自检	4
1.1 通道对照与选用原则	4
1.2 环境自检清单	4
1.3 最小骨架 (官方 Quickstart 模式)	4
2. 会话管理命令	5
2.1 例子：启动、确认暂停、干净收尾	5
3. 运行控制命令	5
3.1 正确编排：go 后轮询/等待暂停，不要用 wait_for_event 等断点	6
3.2 例子：单步 6 条并逐条读寄存器	6
4. 断点命令	6
4.1 例子：符号断点 + 地址断点	6
5. 寄存器读写命令	7
5.1 例子：读 EIP / 改 EIP 重入 add	7
6. 内存读写命令	7
6.1 例子：读堆栈参数 → 改内存 → 验证	7
6.2 例子：二进制补丁 (add → ab, 共 7 字节)	8
7. 反汇编与汇编命令	8
7.1 例子：链式遍历导出整个 add 函数汇编	8
8. GUI 同步命令 (让 x32dbg 看得见)	8
8.1 丝滑原则 (人眼可视三要素)	9
9. 高频编排模式与完整模板	9
9.1 三大实战的命令序列 (final_show.py 的骨架)	9

9.2 换新目标程序：只改三处	9
9.3 排查顺序（断点不命中/读数不对时）	10
10. 避坑速查表	10
附录：客户端 API 与 MCP 工具对照总表（常用部分）	10

1. 总览：两种驱动方式与环境自检

本手册覆盖驱动 x32dbg 的两条通道，命令语义完全一致，只是调用形式不同：

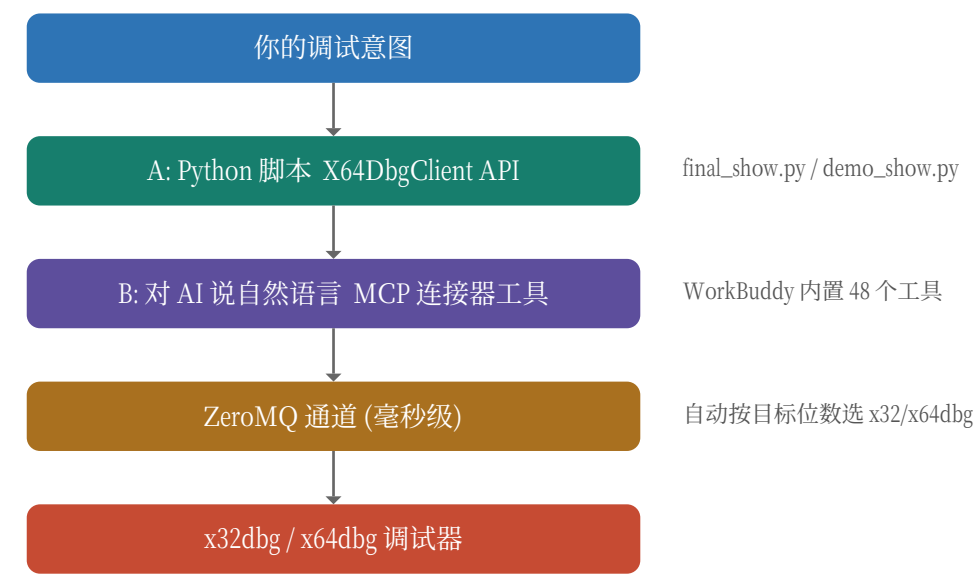


图 1-1 两条调试驱动通道

1.1 通道对照与选用原则

维度	Python 客户端 (X64DbgClient)	MCP 工具 (连接器)
调用形式	脚本 (X64DbgClient)	对 AI 说话 (MCP 工具)
速度特点	批量跑流程, 毫秒/次, 2.5 秒跑完三大实战	一次一个动作, 有 AI 推理往返
适用场景	改脚本顶部 TARGET/断点地址即换目标	适合探索式调试, 边看边问

选用原则：要结果 → 写/改脚本（第 9 章模板三处改动即可复用）；要交互探索或亲眼看过程 → 直接对 AI 说，或用丝滑档 demo_show.py。

1.2 环境自检清单

```
#1)客户端库就位(绿色Python)
E:\out\python312\python.exe -c "import x64dbg_automate; print('client OK')"
```

```
#2)插件就位(两个文件都必须在)
dir E:\out\x64dbg\release\x32\plugins\x64dbg-automate*.dp32
dir E:\out\x64dbg\release\x32\plugins\libzmq-mt-4_3_5.dll
```

```
#3)一键自检(启动->断点->读堆栈->单步全PASS)
E:\out\python312\python.exe E:\out\selftest_xauto.py
```

1.3 最小骨架 (官方 Quickstart 模式)

```
import sys
sys.path.insert(0, r"E:\out\python312\Lib\site-packages")
from x64dbg_automate import X64DbgClient

client = X64DbgClient(x64dbg_path=r"E:\out\x64dbg\release\x32\x32dbg.exe")
client.start_session(r"D:\out\work\func_demo.exe")#启动并加载目标
client.set_breakpoint(0x401530)#先下断点!
client.go()#放行
client.wait_until_stopped()#等到暂停
print(hex(client.get_reg("eip")))#停在哪
client.detach_session()#收尾
```

铁律：一定要先 set_breakpoint 再 go。顺序反了程序会在命中前跑完退出，后续所有读取都会失败。

2. 会话管理命令

Python 客户端 (X64DbgClient)	MCP 工具 (连接器)	中文说明
start_session(target_exe)	start_session	启动调试器并加载目标程序。x64dbg_path 给 x96dbg.exe 时按目标 PE 位数自动选 x32/x64dbg
connect_to_session(pid)	connect_to_session	接管已运行的调试器会话 (传会话 PID)
list_sessions()	list_sessions	列出当前全部调试会话
detach_session()	detach_session	脱离目标继续运行并结束客户端连接 (日常收尾用它)
terminate_session()	terminate_session	结束调试会话并关闭调试器 (清理用)
get_debugger_status()	get_debugger_status	查询状态: 调试中/运行中/暂停, 判断是否停在断点

2.1 例子：启动、确认暂停、干净收尾

```
client = X64DbgClient(x64dbg_path=r"E:\out\x64dbg\release\x96dbg.exe")
client.start_session(r"D:\out\work\func_demo.exe")
client.go();client.wait_until_stopped()#过启动系统断点
print(client.get_debugger_status())#确认paused
...#断点/读内存等操作
client.detach_session()#目标继续跑完, 会话结束
```

坑：① 用 X64DbgClient 直连时给 x96dbg.exe 可能 TimeoutError（启动器子进程 PID 对不上），直连请给 x32dbg.exe / x64dbg.exe 本体，或像 MCP 服务器那样先按位数解析；② 残留的 x32dbg 窗口会占用会话锁，报“启动超时”先查任务管理器关旧进程。

3. 运行控制命令

Python 客户端 (X64DbgClient)	MCP 工具 (连接器)	中文说明
go()	go	放行运行 (从暂停处继续)
wait_until_stopped()	—	阻塞等待调试目标暂停 (官方推荐编排)
stepi(n=1)	step_into	单步执行 n 条指令, 遇 call 跟进函数内部

Python 客户端 (X64DbgClient)	MCP 工具 (连接器)	中文说明
—	step_over	单步越过: 把 call 当一条指令执行, 不跟进
—	run_to_return	执行到当前函数返回 (ret) 处停下
—	trace_into / trace_over	条件跟踪: 批量单步并记录, 可带条件表达式
—	pause	把运行中的目标强制暂停
—	wait_for_event	等调试事件。注意: 会返回队列里积压的陈旧事件, 编排慎用!

3.1 正确编排：go 后轮询/等待暂停，不要用 wait_for_event 等断点

```
#标准写法A(阻塞等待, 推荐)
client.go()
client.wait_until_stopped()#停在断点/异常/退出

#标准写法B(放行到指定地址, 兼容TLS首停)
def go_until(client, addr, max_rounds=6):
    for _ in range(max_rounds):
        client.go(); client.wait_until_stopped()
        if client.get_reg("eip") == addr:
            return
    raise RuntimeError("未能停在0x%X"%addr)
```

最大的坑（血泪）：wait_for_event / MCP 的 wait_for_event 会取出事件队列里积压的陈旧事件（官方文档明示 “ may have occurred before the method was called ”）。拿它编排断点会误以为没到断点而多 go 一次，把停在断点的程序放跑。编排一律用 wait_until_stopped 或轮询 get_debugger_status。

3.2 例子：单步 6 条并逐条读寄存器

```
for i in range(6):
    client.stepi(1)
    client.gui_refresh_views()#每步刷新GUI(第8章)
    print("step %d: EIP=0x%X EAX=%d"
          % (i+1, client.get_reg("eip"), client.get_reg("eax")))
```

4. 断点命令

Python 客户端 (X64DbgClient)	MCP 工具 (连接器)	中文说明
set_breakpoint(addr_or_sym, name=None)	set_breakpoint(address_or_symbol, name)	在地址或符号处下软件断点。符号如 "func_demo.add" 自动解析
clear_breakpoint()	clear_breakpoint	清除断点 (可带地址/名字, 无参清全部)
—	list_breakpoints	列出当前全部断点
—	toggle_breakpoint	启用/禁用某断点 (不删除)

4.1 例子：符号断点 + 地址断点

```

client.set_breakpoint("func_demo.add")#符号断点, 免硬编码地址
client.set_breakpoint(0x401545, name="bp_ret")#地址断点
client.go(); client.wait_until_stopped()
assert client.get_reg("eip")==0x401530#精确停在add入口

```

软件断点(INT3)命中后即被摘除：同一个断点第二次想再命中，必须在放行前重新 set_breakpoint（脚本实战3里 go_until 前重装 ret 断点就是这个原因）。另外重入函数的第二种办法：直接 set_reg("eip", 函数入口)，栈布局没变时等价于重新调用。

5. 寄存器读写命令

Python 客户端 (X64DbgClient)	MCP 工具 (连接器)	中文说明
get_reg("eip")	get_register(register)	读单个寄存器 (eip/esp/ebp/eax/ebx/ecx/edx/esi/edi/eflags...)
get_all_registers()	get_all_registers	一次读全部通用寄存器 (注意: 个别版本实现返回全0, 用单读接口最稳)
set_reg("eip", 0x401530)	set_register(register, value)	写寄存器。经典用法: 改 EIP 重入函数
eval_expression("表达式")	eval_expression(expression)	求值任意调试表达式: 寄存器/符号/地址运算, 如 "func_demo.add+4"

5.1 例子：读 EIP / 改 EIP 重入 add

```

eip = client.get_reg("eip")
esp = client.get_reg("esp")
print("EIP=0x%X ESP=0x%X" % (eip, esp))

#堆栈参数定位: 函数入口断点处, 参数地址=入口时ESP+4/ESP+8
#(此刻EBP还是调用者main的栈帧, 不能用EBP+8!)
client.set_reg("eip", 0x401530)#重入add(栈参数布局未变时)

```

坑：get_all_registers 曾实测返回全 0 —— 排查时以 get_reg("eip") 单读接口为准；入口断点处想拿参数，先读 ESP，再读 [ESP+4]、[ESP+8] (cdecl: a 在 ESP+4, b 在 ESP+8)。

6. 内存读写命令

Python 客户端 (X64DbgClient)	MCP 工具 (连接器)	中文说明
read_dword(addr)	—	读 4 字节整数 (读栈上参数最方便)
read_memory(addr, size)	read_memory(address, size)	按字节读一块内存, 返回 bytes
—	read_memory_many	批量读多段地址 (一次调用省往返)
write_memory(addr, data)	write_memory(address, hex_data)	写内存。脚本传 bytes; MCP 传十六进制串如 "64000000"

6.1 例子：读堆栈参数 → 改内存 → 验证

```
#停在add入口后:读参数a,b(cdecl,参数在返回地址之后)
esp0=client.get_reg("esp")#[esp]=返回地址,[esp+4]=a,[esp+8]=b
a = client.read_dword(esp0 + 4)      # -> 7
b = client.read_dword(esp0 + 8)      # -> 5

#内存补丁:把a改成100(求和指令执行之前改才有效!)
client.write_memory(esp0 + 4, (100).to_bytes(4, "little"))
assert client.read_dword(esp0 + 4) == 100
```

6.2 例子：二进制补丁 (add → ab, 共 7 字节)

```
PATCHES = [
    (0x401536, b"\x8b\x55\x0c"), # mov edx,[ebp+0x0c] -> mov edx,[ebp+8]
    (0x401539, b"\x8b\x45\x08"), # mov eax,[ebp+8]   -> mov eax,[ebp+0xc]
    (0x40153c, b"\x29\xd0"),     # add eax,edx     -> sub eax,edx
]
for addr, data in PATCHES:
    assert client.write_memory(addr, data)
#放行到ret后EAX=2(7-5);该补丁只改内存,重启后消失
```

时机铁律：想让补丁/改内存生效，必须在目标指令执行之前下断改好——程序是顺序执行的，和已经算完再改内存就晚了。

7. 反汇编与汇编命令

Python 客户端 (X64DbgClient)	MCP 工具 (连接器)	中文说明
<code>disassemble_at(addr)</code>	<code>disassemble_at(addr)</code>	反汇编一条指令, 返回 instruction(文本) 和 instr_size(长度)
—	<code>disassemble(address, count)</code>	从地址起批量反汇编 count 条 (一次拿全函数很方便)
—	<code>assemble(address, text)</code>	汇编: 把汇编文本写为目标机器码

7.1 例子：链式遍历导出整个 add 函数汇编

```
def export_disasm(client, addr_start):
    out, a = [], addr_start
    while True:
        ins=client.disassemble_at(a)#一条指令
        out.append("0x%06X %s" % (a, ins.instruction))
        if ins.instruction.startswith("ret"):
            break#函数结束
        a+=ins.instr_size#走到下一条
    return out

for line in export_disasm(client, 0x401530):
    print(line)#补丁前后各导一次做对比
```

8. GUI 同步命令 (让 x32dbg 看得见)

Python 客户端 (X64DbgClient)	MCP 工具 (连接器)	中文说明
gui_refresh_views()	refresh_gui	强制全部视图刷新: 反汇编跟随 EIP, 寄存器/堆栈窗口更新
execute_command("disasm eip")	execute_command(command)	执行任意 x32dbg 命令; "disasm eip" 也能让反汇编窗口跳到 EIP

为什么必须手动刷新：自动化读寄存器/读内存/写内存走 ZeroMQ 直连调试内核，不经过 x64dbg 的 GUI 事件循环——界面对这些操作天然零反应。只有真正的调试事件（断点命中、单步暂停）才会让界面自己动，其余操作想看见必须调刷新。

8.1 丝滑原则（人眼可视三要素）

```
def sync(client):
    client.gui_refresh_views()#1)每个动作后强制重绘

sync(client)
time.sleep(0.6)#2)停顿0.6s, 人眼跟得上
#3)不要把几十个操作挤在一秒里——那是快进电影
```

节奏参考：demo_show.py = 每步刷新 + 单步 0.6 秒 + 阶段 1.2 秒（22 秒看完全程）；final_show.py = 全速（2.5 秒）。丝滑与速度不可兼得，按需二选一。

9. 高频编排模式与完整模板

9.1 三大实战的命令序列（final_show.py 的骨架）

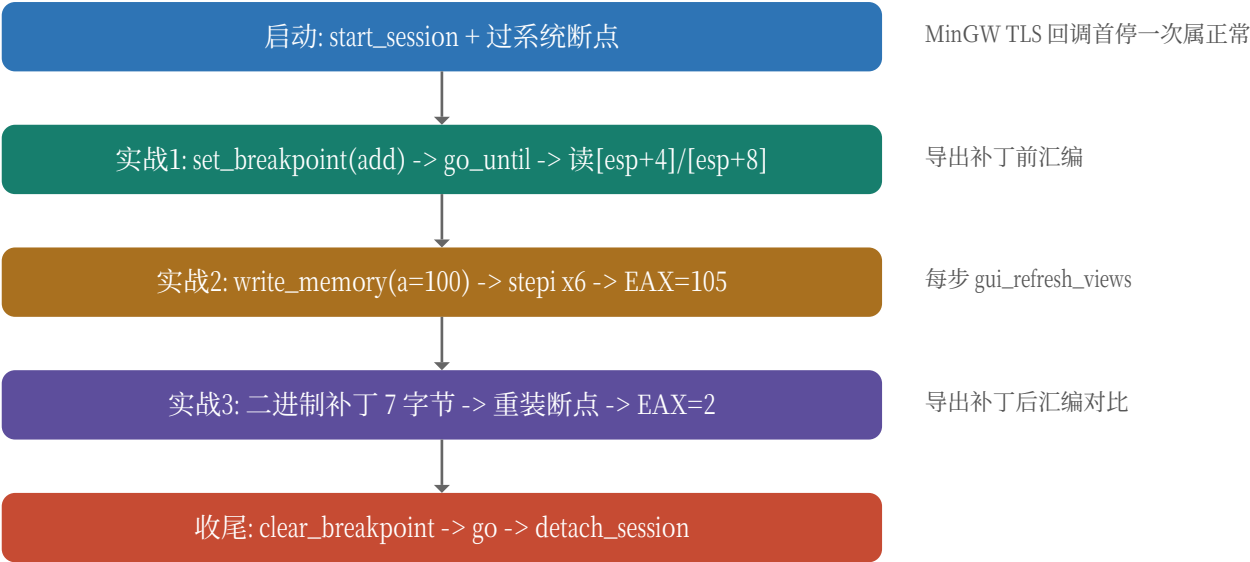


图 9-1 完整调试流程编排

9.2 换新目标程序：只改三处

```
#在final_show.py/demo_show.py顶部:
TARGET=r"D:\out\work\func_demo.exe"#1)换成新程序路径
ADD, RET=0x401530, 0x401545#2)换成你关注的函数地址
PATCHES=[...]#3)换补丁字节(不做补丁可删)
#找函数地址:把程序拖进x32dbg->符号窗口找函数
#或代码里用符号:client.set_breakpoint("你的模块.函数名")
```

9.3 排查顺序（断点不命中/读数不对时）

1. 跑automate_test.py(最小链路验证, 官方Quickstart模式)
2. 跑diag_bp.py(逐步打印断点返回值+每次go后的EIP)
3. 查x32dbg是否残留旧窗口(任务管理器), 关掉重试
4. 确认插件与libzmq-mt-4_3_5.dll在plugins目录
5. 重跑E:\out\selftest_xauto.py看六步是否全PASS

10. 避坑速查表

坑	症状	正确做法
先 go 后下断	程序直接跑完退出, 读啥都失败	永远先 set_breakpoint 再 go
TLS 回调首停	启动后第一次 go 停在 0x4018xx 系统代码	正常现象, 再 go 一次即到用户断点 (go_until 循环兜底)
wait_for_event 陈旧事件	断点“没命中”又多 go 一次, 程序被放跑	编排用 wait_until_stopped / 轮询 get_debugger_status
INT3 命中即摘除	同一断点第二次不命中	放行前重新 set_breakpoint, 或 set_reg 改 EIP 重入
get_all_registers 全 0	读回来全是 0	用 get_reg("eip") 单读接口
EBP 不是当前函数的	EBP+8 读出来的参数不对	入口断点处参数 = 入口时 ESP+4 / ESP+8
GUI 没反应	改了内存/寄存器屏幕没变化	每步调 gui_refresh_views(); 看过程再加停顿
启动超时 TimeoutError	X64DbgClient 直连 x96dbg.exe 超时	直连给 x32dbg.exe/x64dbg.exe 本体; 或让 MCP 服务器解析
会话锁占用	新会话启动卡死	关掉残留 x32dbg 窗口; 客户端自动清 Temp 陈旧锁
import mcp 报错	No module named mcp.server.fastmcp	mcp 必须 pin <2 (2.x 改名 FastMCP); 套装已装好 1.30.0

一句话总纲：先下断再放行；go 后等暂停不数事件；补丁要赶在执行前；想看见就刷新；环境不对先跑 selftest。

附录：客户端 API 与 MCP 工具对照总表（常用部分）

完整 48 个 MCP 工具可在 WorkBuddy 连接器里让 AI 直接列出；下表收录日常调试高频部分。命名规律：MCP 工具名 = 小写下划线，客户端方法名基本一致或近似。

Python 客户端 (X64DbgClient)	MCP 工具 (连接器)	中文说明
start_session / connect_to_session	start_session / connect_to_session	会话: 启动 / 接管
list_sessions / terminate_session	list_sessions / terminate_session	会话: 列出 / 结束

Python 客户端 (X64DbgClient)	MCP 工具 (连接器)	中文说明
get_debugger_status	get_debugger_status	状态查询
go / wait_until_stopped	go	运行控制
stepi(n)	step_into / step_over / run_to_return	单步 / 越过 / 运行到返回
—	trace_into / trace_over	条件批量跟踪
set_breakpoint / clear_breakpoint	set_breakpoint / clear_breakpoint	断点增删
—	list_breakpoints / toggle_breakpoint	断点查改
get_reg / set_reg	get_register / set_register	单寄存器读写
get_all_registers	get_all_registers	全寄存器 (以单读为准)
read_dword / read_memory	read_memory / read_memory_many	内存读
write_memory(addr, bytes)	write_memory(address, hex_data)	内存写
disassemble_at(addr)	disassemble / disassemble_at	反汇编
—	assemble	汇编 (写机器码)
eval_expression	eval_expression	表达式/符号求值
gui_refresh_views()	refresh_gui	GUI 同步刷新
execute_command(cmd)	execute_command	任意 x32dbg 命令透传
—	get_log / get_memory_map / get_modules	日志 / 内存映射 / 模块列表

配套资料：《work目录脚本文件说明.docx》解释 D:\out\work 每个文件；《基于MCP协议的x32dbg自动化调试平台设计与实现.pdf》记录平台完整搭建过程。